

Agent-UCT: Upper Confidence Bounds Applied to Trees for Agentic Workflow Optimization with Cost-Awareness

Yang Li¹, Hai Liu², Dian Shao³, Yu Wang⁴, Xiyu Chen¹,
Sergey Volkov¹, Bozhi Wang⁵, Ziyu Sun⁴, Sihang Liu², Ye Luo^{1,*}, Xiaowei Zhang^{3,*}

¹The University of Hong Kong

²School of Artificial Intelligence, Jiangxi Science and Technology Normal University

³The Hong Kong University of Science and Technology

⁴University of Science and Technology of China

⁵New York University

*Corresponding authors.

Abstract

Optimizing agentic workflows, such as retrieval-augmented generation (RAG) pipelines, requires navigating a combinatorial space of discrete component choices under tight evaluation budgets. Existing approaches—heuristic search, black-box optimization, and standard tree search methods—do not explicitly exploit the compositional structure of these workflows, leading to redundant computation and inefficient budget allocation. We introduce Agent-UCT (Agent-based Cost-Aware Upper Confidence Bounds Applied to Trees), a tree search algorithm that extends UCT with a reuse-aware regularization term derived from a bipartite prefix reuse graph. Agent-UCT biases selection toward branches that leverage previously materialized configuration prefixes, reducing redundant execution while maintaining effective exploration. Our framework, RAGSpace, unifies heterogeneous RAG components from LongRAG, LightRAG, and Self-RAG into a five-dimensional configuration space, enabling systematic cross-framework recombination. WTB (Workflow Test Bench) provides deterministic replay, content-addressable caching, and transactional consistency, ensuring that intermediate states are materialized once and reused across the search. Experiments on HotpotQA and UltraDomain demonstrate that Agent-UCT identifies configurations with the highest out-of-sample performance among the evaluated fixed framework presets. Under full-pool evaluation, bipartite prefix reuse reduces logical search cost by 73.6% relative to the no-prefix-sharing cost upper bound. Compared with full-pool evaluation, sampling-based evaluation further achieves a 4.2× wall-clock speedup. Agent-UCT, RAGSpace, and WTB together provide a unified framework for cost-aware, reproducible, and compositionally efficient agentic workflow optimization.

Introduction

Agentic workflows have emerged as a powerful paradigm for automating complex, multi-step tasks through the orchestration of large language models (LLMs) with external tools, retrieval mechanisms, and decision-making modules. Unlike single-turn prompting, agentic systems exhibit iterative reasoning, stateful execution, and dynamic adaptation—capabilities that are essential for applications such as retrieval-augmented generation (RAG), code assistance, scientific reasoning, and autonomous software agents.

A fundamental challenge in deploying agentic workflows is optimization: selecting the optimal workflow configuration of discrete components that govern agent behavior. In a RAG agent, for example, the workflow comprises components such as chunking strategy, embedding model, retrieval top- k , reranking method, generation prompt, and reflection schedule. Each component offers discrete choices, and the full configuration space grows combinatorially as the product of these options. Even moderately sized pipelines (e.g., 5 components with 3–5 options each) yield hundreds to thousands of candidate configurations.

Evaluating a single agentic workflow configuration is expensive. Each evaluation requires executing the agent across multiple benchmarks or tasks, involving repeated LLM infer-

ence calls, retrieval operations, and tool invocations. A single forward pass can take seconds to minutes, and systematic optimization over thousands of configurations becomes computationally prohibitive. Moreover, agentic workflows are stateful: intermediate states (e.g., retrieved documents, materialized embeddings, conversation history) depend on the execution prefix. When two configurations share a common prefix—such as the same chunking strategy and embedding model—executing that prefix repeatedly across evaluations results in substantial redundant computation.

Existing optimization approaches fall short in addressing these challenges. Heuristic grid search or random sampling ignores the combinatorial structure and wastes evaluation budget on unpromising regions. Black-box optimization methods, including Bayesian optimization and reinforcement learning, treat the workflow as a monolithic function and cannot exploit the compositional reuse of prefixes. Multi-armed bandit algorithms assume independent arms rather than structured dependencies. Standard tree search methods, such as Monte Carlo Tree Search (MCTS) and its UCT (Upper Confidence Bounds Applied to Trees) variant (Kocsis and Szepesvári, 2006; Coquelin and Munos, 2007), are designed for planning domains like games where node evaluations are simulated at low cost and exhibit no computational reuse across branches. In agentic workflow optimization, however,

the cost of evaluating a configuration is path-dependent, and shared prefixes create explicit reuse opportunities that standard UCT ignores.

This paper addresses the problem of discrete agentic workflow optimization under evaluation budgets—that is, finding a near-optimal configuration within a limited number of costly evaluations, where each evaluation can reuse previously materialized prefix states.

We propose Agent-UCT (Agent-based Cost-Aware Upper Confidence Bounds Applied to Trees), a novel tree search algorithm that extends UCT with a reuse-aware regularization term. Agent-UCT operates on a search tree where nodes represent partial workflow configurations (prefixes). The selection policy at each node balances the standard UCT exploration-exploitation trade-off against a *cost regularization term* derived from the marginal execution cost of expanding to a child node. This marginal cost is computed based on a bipartite reuse graph that tracks which configuration prefixes have already been materialized for which benchmark clusters. By favoring branches with lower expected additional cost—i.e., those that reuse previously materialized states—Agent-UCT efficiently allocates evaluation budget toward novel, high-potential configurations while avoiding redundant computation.

Our methodological contributions are threefold:

1. **Cost-aware UCT selection policy with decaying regularization.** We augment the standard UCT score with a regularization term $-\lambda_t \widehat{\Delta C}_t(s, a)$, where $\lambda_t = \lambda_0 / \sqrt{t}$ decays over time, and $\widehat{\Delta C}_t(s, a)$ estimates the marginal computational cost of expanding from state s via action a , conditioned on the current bipartite reuse graph. This biases the search toward branches that leverage previously materialized prefixes while allowing the influence of cost regularization to diminish as reward estimates become more reliable.
2. **Dual-level bipartite reuse graph for fine-grained cost estimation.** We formalize the reuse structure of agentic workflow evaluation at two granularities. At the cluster level, we define a bipartite reuse graph connecting configuration prefixes to benchmark clusters. At the item level, we maintain a refined ledger that tracks prefix materialization at the individual question level. This dual-level structure enables accurate marginal cost estimation under both full-pool and sampling-based evaluation protocols, with the item-level ledger supporting fine-grained reuse tracking during batched evaluation.
3. **Integration with versioned execution and unified configuration space.** Agent-UCT is designed to operate atop WTB (Workflow Test Bench), a reproducible execution backbone that provides deterministic replay, content-addressable caching, and transactional consistency. This integration ensures that reuse estimates are accurate and that each intermediate state is materialized

at most once and reused across the search. We further introduce RAGSpace, a unified framework that standardizes heterogeneous RAG components from LongRAG, LightRAG, and Self-RAG into a five-dimensional configuration space, enabling systematic cross-framework recombination and search.

Related Work

RAG Optimization

Retrieval-augmented generation (RAG) has emerged as a critical paradigm for enhancing large language models with external knowledge (Gao et al., 2023). Recent work has focused on optimizing individual components of the RAG pipeline, including chunking strategies, embedding models, retrieval top-k, reranking methods, and generation prompts.

Common evaluation benchmarks include HotpotQA (Yang et al., 2018) for multi-hop reasoning and UltraDomain (Qian et al., 2025) for domain-specific retrieval. LongRAG addresses multi-hop questions by incorporating both the question and retrieved context into a unified framework (Jiang et al., 2024). Self-RAG introduces iterative retrieval-generation cycles with reflection tokens (Asai et al., 2024), while LightRAG leverages knowledge graph structures for efficient retrieval (Guo et al., 2024).

Despite these advances, most existing approaches optimize RAG components in isolation or through heuristic grid search, which becomes infeasible as the number of components grows. Our work addresses this limitation by framing RAG optimization as a tree search problem over a discrete compositional space, enabling systematic exploration of the full pipeline.

Tree search and cost of transformer

Bandit-based tree search methods have been successfully applied to large-scale planning problems. UCB1 provides a near-optimal solution to the multi-armed bandit problem by balancing exploration and exploitation (Auer et al., 2002). UCT (Upper Confidence Bounds Applied to Trees) extends this principle to tree-structured decision processes (Kocsis and Szepesvári, 2006; Coquelin and Munos, 2007), treating node selection as a local bandit and backpropagating values from simulated rollouts.

However, applying UCT to transformer-based agentic workflows introduces unique challenges. Unlike traditional planning domains (e.g., games), evaluating a complete configuration requires executing expensive transformer inference across multiple benchmarks. The computational cost of a single forward pass can be prohibitive, and similar configuration prefixes often result in redundant computation.

Recent work has explored cost-aware bandit algorithms, but these typically assume fixed per-action costs rather than

path-dependent reuse opportunities. Our Agent-UCT algorithm introduces a reuse-aware regularization term that explicitly accounts for the marginal cost of exploring new branches, leveraging execution caching to avoid redundant computation.

Reproducible Agentic workflow

Reproducibility is a fundamental challenge in agentic workflow research due to the non-deterministic nature of LLM outputs, API dependencies, and stateful execution.

Chatbot Frameworks. General-purpose agent frameworks such as Agno and AutoGen (Wu et al., 2024) provide flexible abstractions for building conversational agents but lack built-in support for deterministic replay or execution caching.

Stateful Workflow Frameworks. LangChain and LangGraph offer directed graph abstractions for composing LLM calls with branching and persistence. However, their state management is primarily designed for application-level checkpointing rather than systematic workflow optimization.

Versioned Execution. AgentGit introduces version control semantics for agentic workflows, maintaining persistent state of both the workspace and the agent through versioned execution (Li et al., 2025). This enables deterministic replay and content-addressable caching of intermediate states. WTB (Workflow Test Bench) extends AgentGit into a reproducible execution backbone, providing transactional consistency and API-level caching for reproducible evaluation.

To our knowledge, no existing framework combines reuse-aware tree search with versioned execution caching for RAG optimization. Agent-UCT and WTB together fill this gap by providing both an efficient search algorithm and a reproducible execution substrate.

Method

We propose Agent-UCT, which extends UCT with a reuse-aware marginal-cost penalty that favors branches whose execution can reuse previously materialized workflow prefixes.

We consider the problem of optimizing an agentic workflow, such as a RAG pipeline, over a discrete compositional search space.

A complete configuration is defined as

$$x = (o_1, o_2, \dots, o_L), \quad o_\ell \in \mathcal{O}_\ell,$$

where ℓ indexes pipeline stages, and o_ℓ denotes the selected option of the component at level ℓ .

The full search space is defined as

$$\mathcal{X} = \mathcal{O}_1 \times \mathcal{O}_2 \times \dots \times \mathcal{O}_L.$$

Let $s_\emptyset = ()$ denote the root state. The space of all partial and complete configuration states is

$$\mathcal{S} = \{s_\emptyset\} \cup \bigcup_{d=1}^L (\mathcal{O}_1 \times \dots \times \mathcal{O}_d).$$

A search state $s \in \mathcal{S}$ at depth d is defined as

$$s = (o_1, \dots, o_d), \quad d \in \{0, 1, \dots, L\}.$$

This compositional structure naturally defines a search tree in which each node represents a partial configuration, and each terminal node represents a complete configuration.

The search space $\mathcal{X}$ grows combinatorially with the number of levels, making exhaustive search unfavorable. Therefore, we build on Upper Confidence Bounds Applied to Trees (UCT), an extension of UCB from multi-armed bandits to tree search. In the standard UCT algorithm, the selection of a child node is treated as a local bandit problem and is governed by a UCB score, and evaluation rewards are back-propagated along the visited tree path, as proposed by Kocsis and Szepesvári (2006).

In RAG scenarios, the evaluation of a complete workflow is expensive, requiring performance assessment over multiple benchmarks. Moreover, the exploration of similar branches often leads to recomputation of shared workflow configuration prefixes.

AgentGit maintains a persistent state of both the workspace and the agent through versioned execution, allowing reuse of previously materialized workflow configuration prefixes.

This motivates Agent-UCT, a cost-aware UCT policy for agentic workflows that incorporates a marginal prefix materialization cost derived from AgentGit-based prefix reuse.

Problem Setup

Search Formulation

A search state s is defined as

$$s = (o_1, \dots, o_d), \quad d \in \{0, 1, \dots, L\}.$$

At depth $d < L$, an action $a \in \mathcal{O}_{d+1}$ selects the component at the next level $d + 1$, and the resulting child state is

$$\text{Child}(s, a) = (o_1, \dots, o_d, a).$$

When $d = L$, the state is terminal and corresponds to a complete configuration: $x_s = s \in \mathcal{X}$.

Benchmark Clusters

A full configuration is evaluated over benchmark clusters, where each cluster consists of a subset of questions drawn from one or more benchmarks:

$$\mathcal{Z} = \{z_1, z_2, \dots, z_K\}.$$

$R(x, z)$ represents the reward of a full configuration x on cluster z . The total reward is calculated as

$$J(x) = \sum_{z \in \mathcal{Z}} w_z R(x, z),$$

where $w_z \geq 0$ and $\sum_{z \in \mathcal{Z}} w_z = 1$.

For equal importance of benchmark clusters,

$$J(x) = \frac{1}{K} \sum_{k=1}^K R(x, z_k)$$

The benchmark clusters allow us to construct a sampling-based evaluation algorithm to estimate the cluster reward from sampled question batches, which further reduces the evaluation cost of Agent-UCT.

WTB, AgentGit and Prefix Reuse

Agent-UCT assumes an execution substrate that supports deterministic prefix materialization and state reuse. In our implementation, this substrate is provided by WTB, which is built upon AgentGit.

WTB serves as the runtime backbone of our framework, providing deterministic execution, content-addressable caching, and transactional consistency for agentic workflow evaluation.

For a full configuration x and a benchmark cluster z , we define $\sigma_m(x, z) = F_m(x_{1:m}, z)$ as the system state, including the workspace and the agent, after executing the first m selected components of x on benchmark cluster z .

This $\sigma_m(x, z)$ is uniquely determined by the configuration prefix $(x_{1:m}, z)$, under the deterministic workflow control of WTB, including API caching and transactional consistency control.

This means that the state is reusable only for the same configuration prefix and benchmark cluster $(x_{1:m}, z)$.

To formalize this structure, we define a bipartite reuse graph

$$G_t^Z = (\mathcal{S}, \mathcal{Z}, \mathcal{E}_t^Z), \quad \mathcal{E}_t^Z \subseteq \mathcal{S} \times \mathcal{Z}.$$

where $\mathcal{S}$ is the set of selected configuration paths, $\mathcal{Z}$ is the set of benchmark clusters, and

$$\mathcal{E}_t^Z \subseteq \mathcal{S} \times \mathcal{Z}$$

represents the materialized $(x_{1:m}, z)$. Also,

$$(x_{1:m}, z) \in \mathcal{E}_t^Z$$

means that $(x_{1:m}, z)$ has already been materialized, and therefore the computation cost of configuration $(x_{1:m})$ for cluster z is waived for any subsequent expansion of child nodes.

We therefore define the marginal search cost of $(d+1)$ -th component as

$$\Delta C_t(s, a; z) = c_{d+1}(s, a, z) \mathbf{1}[(\text{Child}(s, a), z) \notin \mathcal{E}_t^Z].$$

For a minibatch $\mathcal{Z}_t \subseteq \mathcal{Z}$, the aggregated marginal cost estimate is

$$\widehat{\Delta C}_t(s, a) = \sum_{z \in \mathcal{Z}_t} w_z \Delta C_t(s, a; z).$$

UCT with Prefix-State Reuse

As stated above, the cluster-level bipartite reuse graph is defined as

$$G_t^Z = (\mathcal{S}, \mathcal{Z}, \mathcal{E}_t^Z),$$

to estimate the marginal cost of expanding each candidate prefix.

For a nonterminal state s at depth $d < L$, an action $a \in \mathcal{A}(s) = \mathcal{O}_{d+1}$ selects the component at the next level and induces the child prefix

$$s' = \text{Child}(s, a) = (o_1, \dots, o_d, a).$$

If $(s', z) \in \mathcal{E}_t^Z$, then prefix s' has already been materialized for benchmark cluster z , and the corresponding WTB state can be reused. Otherwise, an additional marginal execution cost will be incurred. Thus, Agent-UCT computes the marginal cost estimate from the bipartite reuse graph, denoted by $\widehat{\Delta C}_t(s, a)$.

We define the Agent-UCT score:

$$\begin{aligned} \text{Agent-UCT}_t(s, a) = & \hat{Q}_t(s, a) + c_{\text{UCT}} \sqrt{\frac{\log(N_t(s) + 1)}{N_t(s, a) + 1}} \\ & - \lambda_t \widehat{\Delta C}_t(s, a). \end{aligned} \quad (1)$$

Here, $N_t(s)$ is the visit count of state s , $N_t(s, a)$ is the visit count of edge (s, a) , and $\hat{Q}_t(s, a)$ is the empirical reward estimate of taking action a from state s . The parameter $c_{\text{UCT}} > 0$ is the standard UCT exploration parameter.

The search starts from the root state s_0 . At iteration t , Agent-UCT applies the tree policy in Eq. (1) while the current node is nonterminal and fully expanded.

In full-pool mode, the terminal configuration x_t is evaluated using the objective $J(x_t)$ defined above.

In sampling mode, the evaluator instead returns a sampled reward estimate

$$\hat{J}_t(x_t) = \sum_{z \in \mathcal{Z}} w_z \hat{R}_t(x_t, z),$$

where $\hat{R}_t(x_t, z)$ is computed on the sampled batch $B_{t,z}$. To write the update rule uniformly, define

$$Y_t(x_t) = \begin{cases} J(x_t), & \text{full-pool mode,} \\ \hat{J}_t(x_t), & \text{sampling mode.} \end{cases}$$

After full-pool evaluation, the newly materialized cluster-level reuse keys are

$$\mathcal{M}_t(x_t, z) = \{(x_{t,1:m}, z) : 1 \leq m \leq L, (x_{t,1:m}, z) \notin \mathcal{E}_t^Z\}.$$

The bipartite reuse graph is updated by

$$\mathcal{E}_{t+1}^Z = \mathcal{E}_t^Z \cup \bigcup_{z \in \mathcal{Z}} \mathcal{M}_t(x_t, z).$$

In sampling mode, the corresponding update is performed on the item-level bipartite reuse graph G_t^Q , as defined in the next subsection.

Let τ_t^T denote the tree path from the root state to the expanded node v_t . Agent-UCT backpropagates the task reward $Y_t(x_t)$ along this path. For every state-edge pair $(s_m, a_m) \in \tau_t^T$,

$$\begin{aligned} N_{t+1}(s_m) &= N_t(s_m) + 1, \\ N_{t+1}(s_m, a_m) &= N_t(s_m, a_m) + 1, \end{aligned}$$

and

$$\hat{Q}_{t+1}(s_m, a_m) = \hat{Q}_t(s_m, a_m) + \frac{Y_t(x_t) - \hat{Q}_t(s_m, a_m)}{N_{t+1}(s_m, a_m)}.$$

Note that the Agent-UCT score is used only for tree selection. $\hat{Q}_t$ remains an estimator of the original optimization objective, and $-\lambda_t \widehat{\Delta C}_t(s, a)$ acts as a search-time preference for branches with lower marginal execution cost.

Adaptive Item-Level Cost Estimation

In sampling mode, we further reduce the evaluation cost of Agent-UCT by replacing the cluster-level bipartite reuse graph G_t^Z with a finer-grained item-level bipartite reuse graph.

Let $Q(z)$ denote the fixed question pool within cluster z . At iteration t , the sampled batch from cluster z is

$$B_{t,z} \subseteq Q(z).$$

We define the benchmark-item set as

$$I = \{(z, q) : z \in \mathcal{Z}, q \in Q(z)\}.$$

The item-level bipartite reuse graph is therefore

$$G_t^Q = (S, I, \mathcal{E}_t^Q), \quad \mathcal{E}_t^Q \subseteq S \times I.$$

An edge $(s, (z, q)) \in \mathcal{E}_t^Q$ means that prefix state s has already been materialized for question q in benchmark cluster z before iteration t .

The item-level marginal cost estimate is

$$\begin{aligned} \widehat{\Delta C}_t(s, a) &= \sum_{z \in \mathcal{Z}} w_z \frac{1}{|B_{t,z}|} \sum_{q \in B_{t,z}} c_{d+1}(s, a, z, q) \\ &\quad \times \left(1 - \mathbb{1}[(\text{Child}(s, a), z, q) \in \mathcal{E}_t^Q]\right). \end{aligned} \quad (2)$$

Note that the cost term is computed once per UCT iteration before evaluation. Additional micro-batches drawn before CL convergence do not affect the cost term within the same iteration; they only update the cache state used in subsequent iterations. We also use a decaying regularization coefficient $\lambda_t = \lambda_0/\sqrt{t}$ with $t > 0$. The complete sampling-based Agent-UCT procedure is summarized in Algorithm 1.

Algorithm 1 Sampling-Based Agent-UCT with Item-Level Prefix Reuse

```

1: Initialize search tree  $\mathcal{T}$  with root  $s_\emptyset$ 
2: Initialize  $G_1^Q = (S, I, \mathcal{E}_1^Q)$ , where  $\mathcal{E}_1^Q \leftarrow \emptyset$ 
3: Initialize  $N_1$  and  $\hat{Q}_1$  to zero
4: for  $t = 1, \dots, T$  do
5:   Draw  $\mathcal{B}_t = \{B_{t,z}\}_{z \in \mathcal{Z}}$ 
6:    $s \leftarrow s_\emptyset$ 
7:   while  $s$  is nonterminal and fully expanded do
8:     Compute  $\widehat{\Delta C}_t(s, a)$  for all  $a \in \mathcal{A}(s)$ 
9:      $a^* \leftarrow \arg \max_{a \in \mathcal{A}(s)} \text{Agent-UCT}_t(s, a)$ 
10:     $s \leftarrow \text{Child}(s, a^*)$ 
11:   end while
12:   if  $s$  is nonterminal then
13:     Select an untried action  $a \in \mathcal{A}(s)$ 
14:     Add  $\text{Child}(s, a)$  and edge  $(s, a)$  to  $\mathcal{T}$ 
15:      $s \leftarrow \text{Child}(s, a)$ 
16:   end if
17:   Complete  $s$  to obtain  $x_t \in \mathcal{X}$ 
18:    $(Y_t, \mathcal{M}_t^Q) \leftarrow \text{Eval}(x_t, \mathcal{B}_t, \mathcal{E}_t^Q)$ 
19:    $\mathcal{E}_{t+1}^Q \leftarrow \mathcal{E}_t^Q \cup \mathcal{M}_t^Q$ 
20:   Backpropagate $_{\mathcal{T}}(s, Y_t)$ 
21:   Update  $x_{\text{best}}$  using  $Y_t$ 
22: end for
23: return  $x_{\text{best}}$ 

```

Experiments

We standardize RAGSpace as a five-dimensional configuration space in which chunking, query, retrieval, post-retrieval, and generation are independently searchable. Since LongRAG, LightRAG, and Self-RAG do not all explicitly instantiate these five stages, RAGSpace represents omitted stages using an identity passthrough. This yields a unified five-slot representation for the three reference frameworks, as shown in Table 1.

Through a common contract layer and dependency injection, RAGSpace decouples framework-specific components and enables cross-framework recombination.

Experimental Setup

Datasets: We evaluate Agent-UCT on two benchmarks: HotpotQA (HQ) Yang et al. (2018) and UltraDomain (UD) Qian et al. (2025). HotpotQA evaluates multi-hop question answering over Wikipedia and is grouped into three difficulty levels: easy, medium, and hard. UltraDomain evaluates domain-specific long-form question answering across four domains: agriculture, computer science, legal, and mixed. A 70/30 disjoint split separates each benchmark into a search set (350 items, used by Agent-UCT) and a test set (150 items, reserved for out-of-sample evaluation). Together, these seven spectra cover both fact-centric multi-hop QA and long-form domain-specific QA, providing heterogeneous re-

Preset	1: Chunk 4: Post-Retrieval	2: Query 5: Generation	3: Retrieval
LongRAG	longrag_4k identity	identity longrag_reader	bm25
LightRAG	kg_extraction identity	lightrag_keywords lightrag_answer	lightrag_graph
Self-RAG	standard_passage selfrag_critique	identity selfrag_generator	dense_e5

Table 1: Reference framework presets in the five-dimensional RAGSpace configuration space.

trieval and generation conditions.

Baselines and Search Space: We compare Agent-UCT with fixed LongRAG, LightRAG, and Self-RAG presets represented in the unified RAGSpace space. The full Cartesian product initially defines a 384-point raw configuration space.

Agent-UCT explores cross-framework combinations within this space, whereas each baseline remains confined to its fixed preset.

Evaluation Metrics: For HotpotQA, we report Exact Match (EM) and token-overlap F1 following the official evaluation protocol (Yang et al., 2018). We use F1 as the search reward, while EM is reported as an auxiliary evaluation metric. For UltraDomain, both search reward and out-of-sample evaluation are based on token-level F1 (Qian et al., 2025). For the joint HQ+UD objective, Combined F1 is computed as the arithmetic mean of HQ F1 and UD F1.

Implementation Details: All Agent-UCT experiments use the same backbone model (openai/gpt-5.4-nano), maximum token budget, sampling rule, random seed, and WTB-based execution-reuse infrastructure. All runs include a cold-start phase before the main sampling loop. Implementation details, including run identifiers, wall-clock time, concurrency settings, proxy configuration, and hyperparameters, are reported in supplementary materials.

Configuration Effectiveness and Generalization

Our first research objective is to examine whether Agent-UCT can identify RAG configurations that improve upon fixed framework presets across different evaluation objectives. We therefore evaluate Agent-UCT under three search settings: HotpotQA-only search, UltraDomain-only search, and the combined HotpotQA+UltraDomain search.

Objective-Specific Configuration Discovery

For each objective, we define the *best configuration* as the complete workflow with the highest observed search reward.

Table 2 demonstrates that Agent-UCT selects different configurations across objectives, suggesting that no single fixed pipeline dominates all evaluated settings. Additional

Objective	Best Configurations	Reward
HotpotQA-only	longrag_4k	0.6936
	identity	
	dense_e5	
	selfrag_critique simple_llm	
UltraDomain-only	kg_extraction	0.4241
	lightrag_keywords	
	lightrag_graph	
	lightrag_vector_enrich simple_llm	
HQ+UD	kg_extraction	0.5377
	identity	
	bm25_dense_hybrid	
	selfrag_critique lightrag_answers	

Table 2: Best configurations selected by Agent-UCT under different search objectives.

Dataset	Method	EM	F1	$\Delta F1$	p -value
HotpotQA	Agent-UCT	0.4928	0.5758	–	–
HotpotQA	LongRAG	0.4420	0.5296	+0.0462	0.27
HotpotQA	Self-RAG	0.1812	0.3195	+0.2563	< 0.0001
HotpotQA	LightRAG	0.2536	0.3180	+0.2578	< 0.0001
UltraDomain	Agent-UCT	–	0.3858	–	–
UltraDomain	LongRAG	–	0.3250	+0.0608	< 0.0001
UltraDomain	LightRAG	–	0.2813	+0.1045	< 0.0001
UltraDomain	Self-RAG	–	0.2479	+0.1379	< 0.0001

Table 3: Out-of-sample results for dataset-specific baseline comparison. $\Delta F1$ indicates the performance gain of Agent-UCT over each baseline.

top-ranked configurations from the joint search are reported in the supplementary materials.

Out-of-sample Effectiveness

Table 3 reports the out-of-sample performance of the configurations selected under dataset-specific Agent-UCT search. On HotpotQA, Agent-UCT achieves performance comparable to LongRAG ($p = 0.27$) while significantly outperforming Self-RAG and LightRAG. On UltraDomain, Agent-UCT significantly outperforms all three fixed framework presets ($p < 0.0001$). These results indicate that the configurations discovered by Agent-UCT generalize beyond the search data.

Table 4 reports the out-of-sample performance of the configuration selected under the joint HQ+UD objective. The Agent-UCT selected configuration achieves the highest Combined F1 among the evaluated methods, with statistically significant gains over LightRAG and Self-RAG and a positive gain over LongRAG. These results show that Agent-UCT can identify effective cross-framework configurations that

Method	HQ F1	UD F1	Combined F1	$\Delta F1$	p -value
Agent-UCT	0.5307	0.3776	0.4542	—	—
LongRAG	0.5134	0.3283	0.4208	+0.0334	0.0500
LightRAG	0.3736	0.2855	0.3295	+0.1247	< 0.0001
Self-RAG	0.3195	0.2479	0.2837	+0.1705	< 0.0001

Table 4: Out-of-sample performance of the configuration selected by sampling Agent-UCT under the joint HQ+UD objective.

Metric	Full-Pool 350	Sampling 350
Materialized keys	263,004	112,253
QID score-cache hit rate	78.7%	63.7%
Wall time	157.6h	37.9h (4.2 $\times$ faster)
Total LLM tokens	633.1M	110.4M (5.73 $\times$ fewer)
Chat API calls	228,487	29,056 (7.86 $\times$ fewer)

Table 5: Efficiency comparison between full-pool and sampling.

retain their advantages on data disjoint from the search set. The main out-of-sample results are based on configurations selected under sampling; full-pool results are reported in supplementary materials. The reported p -values are computed using paired two-sided permutation tests, with details provided in the supplementary materials.

Search Effectiveness and Cost Efficiency

Search Efficiency via Sampling

Table 5 shows that Full-Pool 350 achieves a QID hit rate of 78.7%, compared to 63.7% for Sampling 350. Compared with full-pool evaluation, sampling-based evaluation further achieves a 4.2 $\times$ wall-clock speedup and reduces LLM-token consumption and chat API calls by factors of 5.73 and 7.86, respectively.

Table 6 demonstrates that sampling substantially improves search efficiency with limited resource consumption. HotpotQA benefits more from prefix reuse than UltraDomain across all metrics. The embedding cache provides additional savings by avoiding repeated vector computations across search settings.

Efficiency from Bipartite Prefix Reuse

To isolate the effect of bipartite prefix reuse, we compare Agent-UCT’s actual search cost against the theoretical upper bound without prefix sharing, using the same fixed-evaluation protocol and keeping the Layer-1 VDB cache unchanged. Each five-slot configuration induces a sequence of five configuration prefixes of increasing depth. When a candidate shares a prefix that has been materialized by a prior evaluation, the reused portion incurs no additional cost under our logical cost model, thereby reducing the estimated marginal

	Logical Cost (%)	API Calls (k)	API Saving (%)	Chat Tokens (M)	Chat Saving (%)	Emb. Saving (%)
Combined	63.68	71.58	39.56	110.37	59.54	8.81
UltraDomain	55.49	25.07	33.58	76.68	52.40	4.37
HotpotQA	67.58	45.57	43.17	28.83	63.75	11.68

Table 6: Logical costs and resource savings across search settings.

	ANOVA (η^2)		Random Forest (RF)	
Pipeline Slot	Sampling	Full-Pool	Sampling	Full-Pool
generation	0.769	0.831	0.707	0.786
retrieval	0.113	0.145	0.182	0.165
post-retrieval	0.015	0.004	0.080	0.033
chunking	0.006	0.016	0.010	0.001
query	0.002	0.002	0.021	0.015

Table 7: Component-level attribution over evaluated RAG configurations.

cost used by Agent-UCT. We find that prefix reuse reduces the total search cost from 10,920 to 2,878, corresponding to a 73.6% reduction. This result shows that the prefix ledger substantially reduces redundant cost accounting across related RAG configurations. This measurement focuses on logical cost reduction and is not intended to compare the search quality here. Detailed examples and prefix-sharing statistics are provided in the supplementary materials.

Component-Level Attribution Analysis

We further conduct a post-hoc component attribution analysis to identify which pipeline slots drive the reward variation. We report two complementary measures: ANOVA η^2 , which estimates the fraction of reward variance explained by each slot alone, and Random Forest permutation importance, which measures slot-level predictive contribution in an interaction-aware surrogate model.

As shown in Table 7, the relative attribution pattern is consistent in the sampling and full-pool settings. By ANOVA, the **generation** slot has the highest η^2 in both settings, with values of 0.769 and 0.831, respectively. It also receives the highest Random Forest permutation importance in both settings. The **retrieval** slot ranks second under both attribution measures, although its scores are substantially lower than those of **generation**. The remaining slots—**post_retrieval**, **chunking**, and **query**—show comparatively limited attribution scores. Thus, within the evaluated search space and reward setting, reward variation is dominated by **generation**, followed by **retrieval**.

Discussion and Limitations

This work demonstrates WTB-based reuse at the question level and the RAG-workflow level, which is sufficient to show how intermediate states can be traced and reused during Agent-UCT search. However, the current experiments do not fully exercise WTB’s support for more complex file-system-like execution structures. Since WTB adopts a Git-like abstraction, its reuse mechanism can naturally extend from workflow states to finer-grained units such as folders, files, artifacts, and dependency nodes. Future work will explore directory-level and artifact-level caching, with the goal of extending Agent-UCT from RAG pipeline optimization to broader agentic workflow optimization problems.

Our study has several limitations. First, due to the high computational cost of evaluating agentic workflows, we use stratified subsets of HotpotQA and UltraDomain rather than full benchmarks. Second, all experiments use `openai/gpt-5.4-nano` as the backbone LLM, which enables controlled comparison but does not characterize how Agent-UCT behaves across different model capacities. Third, the search space is limited to components drawn from LongRAG, LightRAG, and Self-RAG. While this provides a controlled setting for cross-framework recombination, it does not exhaust the broader design space of RAG components and workflow structures. Finally, UltraDomain requires long-form answers, but our automatic reward uses token-level F1. This metric may be sensitive to answer length and lexical overlap, and may not fully capture factuality or completeness.

Conclusion

We presented a cost-aware framework for optimizing agentic RAG workflows. At the algorithmic level, Agent-UCT performs path-reuse regularized UCT search to identify high-reward RAG configurations under limited evaluation budgets. At the system level, RAGSpace defines a unified five-dimensional configuration space that supports cross-framework recombination of LongRAG, LightRAG, and Self-RAG components. At the evaluation level, WTB supports reproducible workflow execution, intermediate-state caching, and systematic comparison of candidate configurations.

Experiments on HotpotQA and UltraDomain show that Agent-UCT identifies configurations with the strongest out-of-sample performance, outperforming the evaluated fixed presets with statistically significant gains in most comparisons. Sampling reduces measured wall-clock time, token consumption, and API costs, while bipartite prefix reuse reduces search cost. Post-hoc attribution assigns the greatest importance to generation and retrieval across both evaluation protocols. Together, Agent-UCT, RAGSpace, and WTB provide a unified framework for cost-aware search, cross-framework configuration, and reusable workflow execution.

Acknowledgments

We thank the funding support from NSFC-RGC N_HKU7115/22 (T2261160400) and TRS T32-615_24-R.

References

- A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *Proceedings of the International Conference on Learning Representations*, pages 9112–9141, 2024.
- P. Auer, N. Cesa-Bianchi, and P. Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2): 235–256, 2002.
- P.-A. Coquelin and R. Munos. Bandit algorithms for tree search. In *Proceedings of the Twenty-Third Conference on Uncertainty in Artificial Intelligence*, pages 67–74, 2007.
- Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, H. Wang, et al. Retrieval-augmented generation for large language models: A survey, 2023.
- Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang. LightRAG: Simple and fast retrieval-augmented generation, 2024.
- Z. Jiang, X. Ma, and W. Chen. LongRAG: Enhancing retrieval-augmented generation with long-context LLMs, 2024.
- L. Kocsis and C. Szepesvári. Bandit based monte-carlo planning. In *Proceedings of the European Conference on Machine Learning*, pages 282–293, 2006.
- Y. Li, S. Ping, X. Chen, X. Qi, Z. Wang, Y. Luo, and X. Zhang. AgentGit: A version control framework for reliable and scalable LLM-powered multi-agent systems, 2025.
- H. Qian, Z. Liu, P. Zhang, K. Mao, D. Lian, Z. Dou, and T. Huang. MemoRAG: Boosting long context processing with global memory-enhanced retrieval augmentation. In *Proceedings of the ACM on Web Conference 2025*, pages 2366–2377, 2025.
- Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *Proceedings of the First Conference on Language Modeling*, 2024.
- Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018.